Guía Práctica Paso a Paso (Instructor & Lab Manual)

The Localhost Company: Creating a Startup with your Favorite Local AI Models

- ---

  Evento: OpenHUB 2026 (UNAPEC)
- **Duración:** 3 Horas (09:00 AM – 12:00 PM)
- **Stack de Inferencia Local:** Ollama (gemma:e4b, hermes3:8b)
- **Motor de Contenedores:** Podman & Podman Compose (Rootless Containers)
- **Plataformas de Agentes:** Paperclip (Plano de Control), OpenClaw, Hermes Agent y Playwright MCP
- **Roles Operativos:** CEO Agent, Business Analyst Agent, Developer Agent y QA Agent

1. Arquitectura General del Taller

En este laboratorio construiremos una empresa de software autónoma ejecutándose de forma 100% local y segura mediante contenedores *rootless* con **Podman** y modelos servidos directamente desde el *host* con **Ollama**:

1. **Capa de Inferencia Local (Ollama):** Expuesta en el puerto 11434 hacia la red interna de Podman (host.containers.internal), sirviendo gemma:e4b (alta velocidad y estructuración de requerimientos) y hermes3:8b (razonamiento profundo, delegación, uso de herramientas MCP y generación de código).
2. **Capa de Gobernanza y Dirección (Paperclip - CEO Agent):** Actúa como el plano de control donde vive el **CEO Agent**, administra el tablero de tareas, controla el presupuesto de iteraciones y enruta el trabajo entre departamentos.
3. **Capa de Análisis de Negocio (OpenClaw - Business Analyst Agent):** Recibe la iniciativa del CEO y genera el PRD (Product Requirements Document) junto con los criterios de aceptación en formato Gherkin (Given / When / Then).

4. **Capa de Ingeniería (Hermes Agent - Developer Agent):** Conectado vía **Model Context Protocol (MCP)** al repositorio Git local y al sistema de archivos para implementar el código y guardar aprendizajes en su memoria procedimental (~/.hermes/skills/).
5. **Capa de Calidad Autónoma (Hermes Agent + Playwright MCP - QA Agent):** Levanta un navegador *headless* mediante **Playwright MCP**, ejecuta las pruebas E2E basadas en los criterios del Analista y aprueba o devuelve el ticket al CEO con el *stack trace* en caso de error.

Rol Corporativo	Plataforma / Contenedor	Modelo en Ollama	Responsabilidad Principal
CEO Agent	Paperclip Control Plane	hermes3:8b	Desglosa objetivos de la Junta Directiva, asigna tickets por departamento, supervisa el ciclo Dev ↔ QA y controla el límite de reintentos.
Business Analyst	OpenClaw	gemma:e4b	Traduce ideas de negocio en especificaciones técnicas (SPEC.md), define contratos data-testid y redacta escenarios Gherkin.
Developer Agent	Hermes Agent (Dev)	hermes3:8b	Implementa el código en /workspace usando Filesystem y Git MCP, corrige bugs reportados y guarda habilidades aprendidas.

Rol Corporativo	Plataforma / Contenedor	Modelo en Ollama	Responsabilidad Principal
QA Agent	Hermes Agent (QA) + Playwright MCP	hermes3:8b / gemma:e4b	Ejecuta pruebas E2E reales en navegador *headless* contra el código del Dev y emite estado TEST_PASSED o TEST_FAILED con trazas.

2. Fase 0: Preparación del Entorno con Ollama y Podman (09:00 – 09:25)

Paso 0.1: Verificar Podman en modo Rootless y Crear el Workspace

A diferencia de Docker tradicional, **Podman** opera sin un demonio central con privilegios de *root* (*daemonless & rootless*). Esto garantiza que si un agente con acceso a terminal ejecuta un comando inesperado, permanezca estrictamente confinado al espacio de usuario sin comprometer el sistema operativo.

```
# 1. Verificar instalación de Podman y podman-compose
podman --version
podman-compose --version

# 2. Crear la estructura de carpetas para nuestra empresa local
mkdir -p ~/localhost-company/{config,workspace,skills,tests,prompts}
cd ~/localhost-company

# 3. Inicializar el repositorio Git compartido donde trabajará el Developer Agent
git init workspace
```

Paso 0.2: Configurar Ollama y Descargar los Modelos Locales

Configuramos Ollama para escuchar peticiones desde los contenedores de Podman y descargamos los modelos protagonistas del bootcamp:

```
# Permitir que los contenedores de Podman alcancen el servidor Ollama del host
export OLLAMA_HOST=0.0.0.0:11434

# Descargar los modelos del taller
ollama pull gemma:e4b
ollama pull hermes3:8b

# Verificar que ambos modelos estén disponibles
ollama list
```

3. Fase 1: Orquestación de Contenedores con Podman Compose (09:25 – 09:50)

Paso 1.1: Crear el archivo podman-compose.yml

En ~/localhost-company/podman-compose.yml definimos los 5 servicios utilizando el nombre especial host.containers.internal (nativo de Podman para comunicarse con el *host* sin exponer tráfico a internet) y la bandera :Z en los volúmenes para configurar permisos SELinux automáticamente:

```
version: '3.8'

services:
  # 1. Plano de Control y Agente CEO (Paperclip)
  paperclip-hq:
    image: paperclip-ai/paperclip:latest
    container_name: localhost-ceo-hq
    ports:
      - "3000:3000"
    environment:
      - OLLAMA_BASE_URL=http://host.containers.internal:11434/v1
      - DEFAULT_MODEL=hermes3:8b
```

```
    - COMPANY_CONFIG=/app/config/company.yaml
  volumes:
    - ./config:/app/config:Z
    - ./prompts:/app/prompts:Z
    - ./workspace:/app/workspace:Z
  extra_hosts:
    - "host.containers.internal:host-gateway"

# 2. Agente Analista de Negocios (OpenClaw con gemma:e4b)
analyst-openclaw:
  image: openclaw/openclaw:latest
  container_name: localhost-analyst
  ports:
    - "4000:4000"
  environment:
    - LLM_PROVIDER=ollama
    - OLLAMA_API_BASE=http://host.containers.internal:11434
    - MODEL_NAME=gemma:e4b
    - PAPERCLIP_API_URL=http://paperclip-hq:3000/api
    - AGENT_ROLE=business_analyst
  volumes:
    - ./prompts:/app/prompts:Z
    - ./workspace:/app/workspace:Z
  extra_hosts:
    - "host.containers.internal:host-gateway"

# 3. Agente Desarrollador (Hermes Agent con hermes3:8b)
dev-hermes:
  image: nousresearch/hermes-agent:latest
  container_name: localhost-developer
  ports:
    - "8080:8080"
  environment:
    - OPENAI_BASE_URL=http://host.containers.internal:11434/v1
    - OPENAI_API_KEY=ollama-local
    - HERMES_MODEL=hermes3:8b
    - PAPERCLIP_API_URL=http://paperclip-hq:3000/api
    - AGENT_ROLE=developer
    - MCP_FILESYSTEM_ROOT=/workspace
```

```
    volumes:
      - ./workspace:/workspace:Z
      - ./skills:/root/.hermes/skills:Z
      - ./prompts:/app/prompts:Z
    extra_hosts:
      - "host.containers.internal:host-gateway"

  # 4. Servidor Playwright MCP (Navegador Headless para Pruebas E2E)
  playwright-mcp:
    image: playwright-mcp-server:latest
    container_name: localhost-playwright-mcp
    ports:
      - "8931:8931"
    command: ["npx", "@playwright/mcp@latest", "--headless", "--port", "8931"]

  # 5. Agente QA Tester (Hermes Agent conectado a Playwright MCP)
  qa-hermes:
    image: nousresearch/hermes-agent:latest
    container_name: localhost-qa
    environment:
      - OPENAI_BASE_URL=http://host.containers.internal:11434/v1
      - OPENAI_API_KEY=ollama-local
      - HERMES_MODEL=hermes3:8b
      - PAPERCLIP_API_URL=http://paperclip-hq:3000/api
      - AGENT_ROLE=qa_engineer
      - MCP_SSE_URL=http://playwright-mcp:8931/sse
    volumes:
      - ./workspace:/workspace:Z
      - ./tests:/tests:Z
      - ./prompts:/app/prompts:Z
    depends_on:
      - playwright-mcp
    extra_hosts:
      - "host.containers.internal:host-gateway"
```

4. Fase 2: Configuración del Organigrama en Paperclip (09:50 –

10:25)

Paso 2.1: Crear la Gobernanza Corporativa (config/company.yaml)

En ~/localhost-company/config/company.yaml establecemos la jerarquía donde el **CEO Agent** supervisa a los 3 agentes operativos y automatiza el ciclo de retroalimentación entre Desarrollo y QA:

```
company:
  name: "The Localhost Company"
  max_fix_iterations: 3
  require_board_approval_on_release: true

roles:
  ceo:
    title: "Chief Executive Officer"
    model: "hermes3:8b"
    prompt_file: "/app/prompts/ceo.md"
    subordinates:
      - business_analyst
      - developer
      - qa_engineer

  business_analyst:
    title: "Lead Business Analyst"
    endpoint: "http://analyst-openclaw:4000"
    model: "gemma:e4b"
    prompt_file: "/app/prompts/analyst.md"
    deliverables:
      - "/workspace/SPEC.md"

  developer:
    title: "Senior Full-Stack Developer"
    endpoint: "http://dev-hermes:8000"
    model: "hermes3:8b"
    prompt_file: "/app/prompts/developer.md"
    mcp_servers:
      - filesystem
      - git
```

```
    - bash_runner

  qa_engineer:
    title: "Autonomous QA Automation Engineer"
    endpoint: "http://qa-hermes:8000"
    model: "hermes3:8b"
    prompt_file: "/app/prompts/qa.md"
    mcp_servers:
      - playwright_mcp

workflow:
  stages:
    - id: "1_ANALYSIS"
      owner: "business_analyst"
      next_on_success: "2_DEVELOPMENT"
    - id: "2_DEVELOPMENT"
      owner: "developer"
      next_on_success: "3_QA_TESTING"
    - id: "3_QA_TESTING"
      owner: "qa_engineer"
      next_on_success: "4_CEO_FINAL_REVIEW"
      next_on_failure: "2_DEVELOPMENT" # Rebote automático al Developer con el stack trace
    - id: "4_CEO_FINAL_REVIEW"
      owner: "ceo"
      next_on_success: "BOARD_SIGN_OFF"
```

(10:25 – 10:40: Receso de 15 minutos para café, networking y verificación de contenedores en las laptops de los participantes).

5. Fase 3: Programación de los 4 System Prompts (10:40 – 11:10)

Creamos los archivos de instrucciones dentro de ~/localhost-company/prompts/ para que cada agente conozca sus responsabilidades exactas y el contrato de comunicación:

1. prompts/ceo.md (CEO Agent - hermes3:8b)

Eres el CEO de "The Localhost Company". Reportas a la Junta Directiva (el usuario humano).

Tu misión es transformar objetivos de negocio en software funcional y verificado sin intervención humana intermedia.

REGLAS DE ORQUESTACIÓN:
1. Al recibir una directiva de la Junta, asigna la tarea al `business_analyst` para que genere `/workspace/SPEC.md`.
2. Verifica que `SPEC.md` contenga criterios de aceptación Gherkin y atributos `data-testid` claros. Luego asigna la implementación al `developer`.
3. Cuando el `developer` indique que el código está desplegado en `/workspace`, ordena al `qa_engineer` ejecutar la suite E2E con Playwright MCP.
4. Si el `qa_engineer` devuelve `STATUS: TEST_FAILED`, reasigna el ticket al `developer` adjuntando el log de error exacto. Tienes un presupuesto máximo de 3 reintentos antes de escalar a la Junta Directiva.
5. Cuando el `qa_engineer` devuelva `STATUS: TEST_PASSED`, redacta un resumen ejecutivo para aprobación final de la Junta Directiva.

2. prompts/analyst.md (Business Analyst Agent - gemma:e4b en OpenClaw)

Eres el Business Analyst de "The Localhost Company".
Conviertes ideas de alto nivel del CEO en especificaciones técnicas accionables y sin ambigüedades.

ESTRUCTURA OBLIGATORIA DE `/workspace/SPEC.md`:
1. Descripción funcional y reglas de negocio matemáticas/lógicas.
2. Mapa de Selectores DOM Obligatorios (`data-testid`): Especifica los IDs exactos que el Developer debe colocar en el HTML para que el QA Agent pueda interactuar con ellos sin selectores frágiles.
3. Escenarios de Prueba en formato Gherkin (`Feature`, `Scenario`, `Given`, `When`, `Then`) cubriendo el camino feliz (Happy Path) y casos borde (Edge Cases).

3. prompts/developer.md (Developer Agent - hermes3:8b en Hermes Agent)

Eres el Senior Developer de "The Localhost Company".
Operas sobre `/workspace` mediante tus herramientas Filesystem MCP, Git MCP y Terminal.

REGLAS DE INGENIERÍA:
1. Lee cuidadosamente `/workspace/SPEC.md` antes de escribir código.
2. Implementa la solución respetando estrictamente todos los atributos `data-testid` definidos por el Business Analyst.
3. Si recibes un ticket rebotado con `STATUS: TEST_FAILED`, lee el reporte de Playwright MCP, identifica el bug

en el código, corrígelo y realiza un nuevo commit en Git.
4. Al completar el ticket satisfactoriamente, sintetiza el patrón técnico aprendido en un archivo Markdown dentro de `~/.hermes/skills/` para futuras tareas.

4. prompts/qa.md (QA Agent - hermes3:8b + Playwright MCP)

Eres el Lead QA Automation Engineer de "The Localhost Company".
Estás conectado a un navegador real en modo headless a través de Playwright MCP.

PROTOCOLO DE VALIDACIÓN:
1. Lee los escenarios Gherkin y los selectores `data-testid` en `/workspace/SPEC.md`.
2. Utiliza las herramientas de Playwright MCP (`browser_navigate`, `browser_fill`, `browser_click`, `browser_snapshot`) para probar la aplicación en vivo.
3. Compara el resultado real del DOM contra el resultado esperado en cada escenario Gherkin:
  - Si algún escenario falla: Responde al CEO con `STATUS: TEST_FAILED`, indicando el escenario, el valor esperado, el valor obtenido y el snapshot del DOM.
  - Si todos los escenarios pasan: Guarda la prueba automatizada en `/tests/regression.spec.ts` y responde al CEO con `STATUS: TEST_PASSED`.

6. Fase 4: Simulación en Vivo – El Ciclo Completo de la Empresa (11:10 – 11:45)

Paso 4.1: Iniciar todos los Agentes con Podman

```
cd ~/localhost-company
podman-compose up -d
podman ps
```

Paso 4.2: Enviar el Proyecto desde la Junta Directiva al CEO

Los participantes abren el dashboard de Paperclip en http://localhost:3000 (o usan la terminal) para enviar la orden de negocio a su empresa local:

```
curl -X POST http://localhost:3000/api/board/directive \
 -H "Content-Type: application/json" \
 -d '{
  "project": "Módulo de Checkout con Validador de Cupones",
  "directive": "Crear una aplicación web de checkout con un precio base de
100. Si el usuario ingresa el cupón OPENHUB20 se aplica un 20% de descuento (80 final). Si ingresa un cupón inválido
debe mostrarse una alerta de error y mantener el total en $100."
 }'
```

Paso 4.3: Trazabilidad Paso a Paso Durante la Demo

1. **Delegación del CEO (hermes3:8b):** El CEO registra el proyecto y asigna la tarea #1 al contenedor localhost-analyst.
2. **Entrega del Business Analyst (gemma:e4b):** En segundos, OpenClaw genera /workspace/SPEC.md definiendo los atributos data-testid="coupon-input", data-testid="apply-btn", data-testid="total-amount" y data-testid="error-banner", junto con 2 escenarios Gherkin.
3. **Construcción del Developer (hermes3:8b):** Hermes Agent lee SPEC.md, crea index.html y checkout.js, hace el commit inicial en Git y expone la aplicación en el puerto 8080.
4. **Pruebas E2E del QA Agent (Playwright MCP):** El agente QA abre la URL interna con Playwright MCP, ingresa OPENHUB20, hace clic en aplicar y verifica el DOM.
5. **Demostración del Rebote Autónomo (Self-Healing Workflow):**
    - Para demostrar el poder de la arquitectura, introducimos un bug deliberado en checkout.js (por ejemplo, que el descuento reste $30 en vez de $20).
    - Al correr el QA Agent, Playwright MCP detecta Expected: $80, Received: $70 y emite STATUS: TEST_FAILED.
    - El **CEO Agent** recibe el rechazo y sin intervención humana le ordena al **Developer Agent** reparar la fórmula.
    - El Developer corrige la línea en checkout.js, el QA vuelve a ejecutar Playwright MCP, obtiene STATUS: TEST_PASSED y el CEO solicita la firma final de la Junta Directiva.

7. Fase 5: Inspección de Memoria Procedimental y Cierre (11:45 – 12:00)

Para cerrar el bootcamp, los participantes inspeccionan cómo su empresa local ha aprendido de la experiencia y cómo operarla de forma permanente en Linux:

```
# 1. Ver las habilidades (skills) que Hermes Agent compiló automáticamente en disco
ls -la ~/localhost-company/skills/
cat ~/localhost-company/skills/checkout-coupon-validation.md

# 2. Ver el historial de commits generados por los agentes
cd ~/localhost-company/workspace && git log --oneline

# 3. (Opcional) Convertir el pod de Podman en un servicio systemd de usuario en Linux
podman generate systemd --new --name localhost-ceo-hq > ~/.config/systemd/user/localhost-company.service
```